

Business process runtime models: towards bridging the gap between design, enactment, and evaluation of business processes

Anat Goldstein¹  · Thomas Johanndeiter² · Ulrich Frank²

Received: 9 July 2017 / Revised: 6 April 2018 / Accepted: 29 May 2018 / Published online: 7 June 2018
© Springer-Verlag GmbH Germany, part of Springer Nature 2018

Abstract Business process management (BPM) broadly covers a lifecycle of four distinct phases: design, configuration, enactment, and analysis and evaluation. Most BPM tool suites impose a strict separation between these phases, i.e., in each phase different languages and tools are used and the transition between phases is indirect and costly. This paper presents an environment for integrating all phases of the BPM lifecycle in which business process (BP) types and their instances can be modeled, visualized, managed and automatically synchronized, using a shared representation of models and code. The environment extends the capabilities of BP models to be used not only for specifying BPs but also for: (1) enactment—creating instance objects that capture BP operational data; (2) monitoring BP instances as they progress; (3) visualizing performance indicators of executed BPs at runtime; and (4) navigating from a BP type model to its respective instance population. As opposed to existing tools, the proposed environment does not require regenerating the workflow schema when BP designs change, nor does it require additional adaptations to support monitoring. Thereby, we facilitate a continuous and dynamic BPM environment, where workflow specifications can be changed at runtime. Our solution integrates a meta-programming language called eExecutable Modeling Facility (XMF) and the multi-perspective enterprise modeling framework (MEMO).

Keywords Business process modeling · Runtime models · Process monitoring · Workflow management systems · Enterprise modeling · Continuous BPM

✉ Anat Goldstein
anatgold@tauex.tau.ac.il

¹ Department of Technology and Information Management, Tel Aviv University, P.O. Box 39040, 6997801 Tel Aviv, Israel

² Information Systems and Enterprise Modeling, Institute for Computer Science and Business Information Systems, University of Duisburg-Essen, Essen, Germany

1 Introduction

Business process management (BPM) is defined as “supporting business processes (BP) using methods, techniques, and software to design, enact, control, and analyze operational processes involving humans, organizations, applications, documents, and other sources of information” (van der Aalst et al. 2003). Correspondingly, a BPM system is a software system that is driven by BP designs and used for supporting, automating, and managing routine BPs in organizations (van der Aalst et al. 2003).

BPM broadly covers a lifecycle of four distinct phases: process design, system configuration, enactment, and analysis and evaluation (Weske 2012; van der Aalst et al. 2003). Process design is aimed at describing and validating business processes types, resulting in the creation of BP models. In the system configuration phase, BP designs are implemented in a BPM system (e.g., a workflow management system that defines and manages the execution of workflows). In the enactment phase, operational BP instances are executed and controlled using the configured system. Finally, in the analysis and evaluation phase information on executed operational BPs, usually recorded in execution logs, is analyzed in order to evaluate BP performance and to identify potential improvements and necessary adaptations of the process execution specification. The four phases of the lifecycle and the outputs of each phase are presented on the left side of Fig. 1.

Business processes are pivotal elements of organizational action and decision making and thus managers need to be able to follow them throughout the phases of the lifecycle. First, they need to know how a business process type is defined. For example, a customer order is accepted only after the customer’s credibility has been checked. In addition, it should be possible for them to adapt a type to changing requirements. For example, it should be added that there is no need for a credibility check, if the total sum of an order is lower than a certain value. Furthermore, managers need to know characteristics of particular process instances and of sets of process instances. For example, when did a particular process start

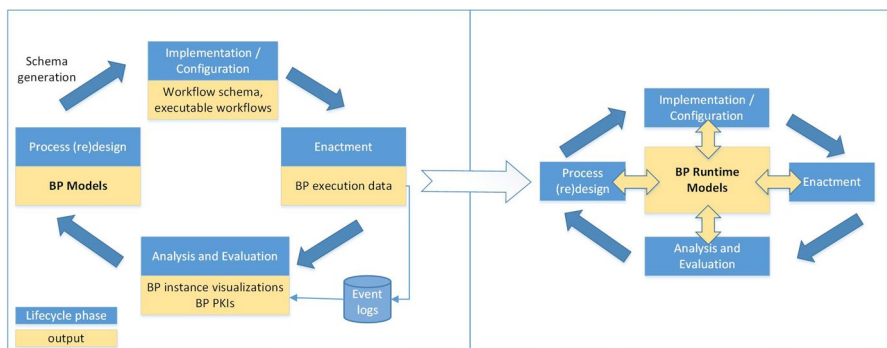


Fig. 1 The left side presents the phases of the BPM lifecycle and the outputs of each phase. The output of each phase serves as the input for the next step. On the right the proposed solution is illustrated, showing that all phases are conducted using BP models

and terminate? What was the average execution time of all processes of a certain type in 2016?

However, currently, in most BPM suites aimed at supporting the entire lifecycle (e.g., YAWL—Yet Another Workflow Language, Camunda, and jBPM) there is a strict separation between the different lifecycle phases, which is imposed by the different representations of BPs at the different phases (Bandara et al. 2007; Rangiha et al. 2015; Hull and Nezhad 2016). This separation is depicted in the left side of Fig. 1. More specifically, for BP design we employ a process modeling tool in order to capture the business specification of a BP in a BP model, using a selected process modeling language, e.g., Business Process Model and Notation—BPMN 2.0 (Object Management Group 2011). The resulting BP model is refined with technical details for workflow execution (e.g., interfaces to other operational systems) and compiled into an executable workflow definition. Such a workflow definition is expressed with a formal language supported by the BPM system's workflow engine, e.g., XML Process Definition Language—XPDL (WfMC 2012) or Web Services Business Process Execution Language or in short, WS-BPEL (OASIS 2007). With this workflow definition, the BPM system is able to execute a process in its runtime environment according to said pre-defined, i.e., static, workflow structure. Consequently, the runtime process instances are not connected to the BP model defined in the design phase. Thus, when the execution of a BP has to be adapted in order to react to new business requirements, we either need to anticipate all possible changes at build-time or we need to modify the original (build-time) BP model correspondingly. Transferring the BP model change to the workflow engine then requires recompiling the model into the workflow definition format and loading the new workflow definition to the engine (Capra and Cazzola 2012; Qiu and Wong 2007; Weske 2012).

While the outlined separation of build time and runtime environments worked sufficiently well for static, routine BP environments, such separation is problematic in dynamic environments, where BPs are subject to regular changes (Qiu and Wong 2007; Capra and Cazzola 2012). Changes in BP designs may be required for several reasons, for example, when at design time the workflow specification is incomplete due to lack of knowledge, when errors or exceptional situations are identified during the workflow execution and require deviating from the static schema, or due to changes in the business environment.

In addition to the dissatisfactory separation of build-time and runtime described above, there is a separation of runtime (i.e., the enactment phase) and evaluation-time (i.e., the analysis and evaluation phase). Most BPM systems record BP executions to log files (event logs). These log files are analyzed with respect to BP performance indicators. Analyzing such log files, usually represented in text format, requires first parsing and converting them into a formal data structure (van der Aalst 2011). This situation is not satisfying because the evaluation is done in retrospect, it is time consuming, and it requires dedicated tools external to the BPM system (e.g., IBM FileNet Case Analyzer 2016). Nevertheless, in recent years, tools emerged which support nearly real-time monitoring and analysis of BP executions (e.g., IBM's Business Monitor 2017). While such tools indeed shorten the time between execution and analysis, a substantial effort is required for transferring BP instance data into objects of the monitoring environment, as it is separated from the runtime

environment. For example, all types of events within a BP are represented using a general ‘event’ class. Deriving statistics of different event types requires processing the different event objects and distinguishing between their different types.

The separation of BPM lifecycle phases additionally finds its expression in the different visualizations of BPs that are used at each phase. Visual representation of runtime information on process executions often diverges from those of design time BP models. Moreover, visualizations of process instances employed in BPM systems are deemed insufficient for process managers (Bandara et al. 2007). Hence, BPM would benefit from consistently using visual renderings of BPs used in the design phase. As the design phase primarily addresses business users, these visualizations are assumed to be best tailored to process analysts.

We argue that the described problems—difficulty to synchronize the evolution of build-time models, enactment (runtime) environment, and analysis and evaluation environment as well as insufficient BP monitoring and analysis support—stem from the fact that in each phase of the lifecycle different (formal and non-formal) representations of business process specifications are used (Ko et al. 2009), as can be seen in the yellow boxes at the left side of Fig. 1.

To overcome these problems, an integration of all phases of the BP lifecycle using a shared BP representation is called for, which allows for visualizing, managing, and automatically synchronizing both BP types and their instances.

The research presented in this paper is aimed at such an integration. A comprehensive modeling environment is developed that supports modeling, enactment, monitoring, and analysis of business processes. The environment is based on BP models, which have so far been proven useful for representing BP types. It extends the capabilities of BP models so that they can be used not only for representing BP specifications but also for: 1. creating instance level objects that capture BP operational data at runtime, i.e., BP models themselves are used for enactment (no translation into a different workflow definition language is required); 2. visualizing BP instances as they progress (we call these BP instance models) using similar graphical notations that are used for BP type models; 3. visualizing performance indicators and other characteristics of sets of executed BPs, for example visualizing the average duration of each BP activity based on the history of executions; and 4. navigating from a BP type model to its respective instance population (represented in instance models) and vice versa at runtime.

We call BP models enhanced with these capabilities BP Runtime Models. As opposed to existing tools, the proposed environment does not require regenerating or re-implementing the workflow schema when BP designs change, nor does it require manual adaptations or additional processing to support monitoring. Instead, the environment (re)uses the same BP models in all phases, as illustrated in the right side of Fig. 1. Thereby we avoid the problematic separation of artifacts across the different lifecycle phases, contribute to reusability and flexibility of BPs, and facilitate a truly continuous BPM environment, where workflow specifications (in the form of BP type models) can be changed at runtime and immediately change the runtime environment’s code.

The implementation of a modeling environment that supports such capabilities creates challenges, which can hardly be overcome with conventional object-oriented

(OO) programming languages (e.g., Java and C#), the prevalent programming paradigm used for realizing BP modeling tools and workflow engines. In Sect. 4 we discuss these challenges, which relate to the support of multiple, adaptable levels of classification in software. In addition, we define requirements for the targeted modeling environment.

In this study, we present a novel approach, which enables overcoming these challenges, by exploiting a meta-programming language called eXecutable Modeling Facility (XMF, Clark et al. 2008). XMF is a multilevel language that enables an arbitrary number of classification levels. This additional abstraction not only contributes to reusability and flexibility, but also allows for instantiating (and executing) classes within the modeling tool as detailed in Sect. 5. We use XMF to implement the BP modeling language, which is part of a method for multi-perspective enterprise modeling (MEMO) (Frank 2012). This XMF-based BP modeling language allows for creating BP (type) models that are integrated with their instances. Furthermore, the use of an enterprise modeling framework for creating process models enables describing the context in which BPs are executed, e.g., IT resources that are used, responsibilities of organizational units, and even business goals addressed by the BP.

Our results are evaluated in Sect. 6. First, we demonstrate through a use case how the resulting environment allows for creating a BP model, executing the model, and monitoring its executions. Since our prototypical environment does not yet offer convenient interfaces for users (e.g., employees) and for software applications that execute process activities, we created an agent that imitates external events and initiates the execution of BP activities based on BP models and predefined event probabilities. Second, our approach is evaluated with respect to objectives and requirements, which are defined in Sect. 4. Third the proposed artifact is evaluated by a panel of experts, who in general agree upon its contribution to continuous BPM. Finally, conclusions and an outlook are presented in Sect. 7.

In the next section, we provide some background on MEMO, on BP types and instances, and on runtime models and then present related work in Sect. 3.

2 Background

In this section, we present the MEMO framework, which provides the BP modeling language for our research; we then explain the difference between BP types and instances. Finally, the concept of a runtime model is presented and defined in the context of our objectives.

2.1 BP modeling with MEMO

In this work, we use MEMO OrgML as a BP modeling language. MEMO is a multi-perspective enterprise modeling method based on a high-level framework structuring the enterprise from three generic perspectives: strategy, organization, and information system, each of which can be further detailed into various aspects

(e.g., resource, structure, process, security, and goal). To allow for more elaborate analysis, each perspective is associated with a set of diagram types. Each diagram type is supported by a domain specific modeling language (DSML).

Among the various languages of MEMO is OrgML, a DSML for organizational modeling (Frank 2011a). OrgML incorporates the core concepts of BP modeling: (sub-) processes (i.e., activities), events, and exceptions. A set of control flow concepts specifies the orchestration of processes, e.g., sequence and branching. In addition, OrgML offers a visual notation that caters to the business domain (see Fig. 2).

The semantics and abstract syntax of all MEMO DSMLs are specified using a meta-modeling language—MML (Frank 2011b). MML is tailored for designing DSMLs that have some unique advantages over other modeling languages. Most notably, MML supports intrinsic features. With intrinsic features, a MEMO DSML can include meta-concepts, which are not relevant for the types of a model created with the DSML (i.e., instances of meta-concepts), but only for instances of these types. In other words, intrinsic features allow to include instance level abstractions within the metamodel, making them suitable for representing both

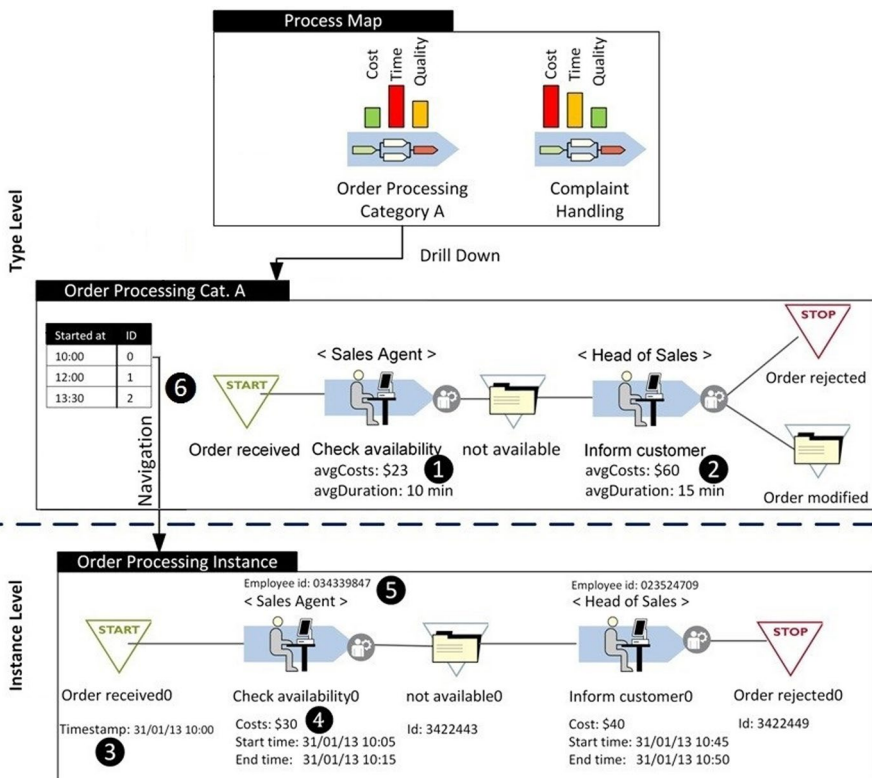


Fig. 2 Order processing business process type model and a corresponding instance model

types and instances. Thus, intrinsic features create a strong motivation for using OrgML in the proposed solution, as further discussed in Sect. 5.3.

Sharing a common metamodel facilitates the integration of MEMO DSMLs. Therefore, OrgML is not limited to describing business process models or organizational structures; it is integrated with DSMLs targeting other perspectives of the enterprise. Consequently, OrgML can reference concepts describing the IT perspective (ITML, Frank et al. 2009), the strategy perspective (GoalML, Overbeek et al. 2015), as well as inter-perspective aspects such as IT security and risks (Goldstein and Frank 2016). For example, a subprocess can be associated with required IT resources, costs, risks, and strategic goals. Because of this integration, OrgML's BP models strongly contribute to the objectives of BPM, as perspectives of various stakeholders can be accounted for to facilitate comprehensive business analysis and improve decision making. In addition to that, the MEMO OrgML offers a visual notation that caters especially to the business domain (Frank 2011a), compared to other BP modeling languages (e.g., BPMN and Event-driven Process Chain—EPC) that have more generic notations.

The specification of MEMO is freely available and documented in several publications. MEMO is accompanied by a modeling tool—MEMO Center, which is implemented in Java within the Eclipse Modeling Framework (EMF) (Eclipse 2017).

2.2 Business process types and instances

Business process models are widely accepted as key abstraction of any organization, as they are used for representing (literally) how an organization works at the operational level. In particular, BP models contribute to analyzing and eventually improving the efficiency of the organization. Simultaneously, they provide a conceptual foundation for designing software systems that support executing business processes (Frank 2011a). BP models typically comprise the domain-specific concepts Subprocess (also known as Activity) and Event, which are then extended and differentiated into more specific concepts (Schonenberg et al. 2008; Frank 2011a; van der Aalst 2013; Weske 2012).

Usually, business process models refer to what is known as the type level. They describe types or classes of business processes (e.g., processing of an order, procurement of supplies, etc.) rather than particular instances (e.g., a processing of a specific order that occurred at a certain time). This is illustrated in Fig. 2: the upper part of the figure depicts a business process map, i.e., a collection of existing BPs, and a drill-down to a particular BP model, which classifies an organization's set of possible order processing occurrences. The lower part of the figure depicts a runtime execution of such an order processing that occurred at a certain point in time, that is, an instance of the BP. In fact, the bottom part of Fig. 2 represents an instance model, i.e., it abstracts from a particular BP instance of the BPM system, visualizing features that are important for the user (Gonzalez-Perez and Henderson-Sellers 2007; Kühne 2006).

So far we mentioned two levels of abstraction: the instance level, which is referred as M0, and the type level, which is referred as M1. However, using concepts at M1 requires their definition on the metamodel level—M2 (in our case—the concepts defining a BP). Respective concepts need to be defined on the meta-metamodel level—M3 (Frank 2011b). While this could result in a *regressus ad infinitum*, the concepts at M3 are often taken as the ultimate reference—a view that is especially promoted by the OMG with the Meta-Object Facility (MOF 2011). For example, the UML is defined by the MOF and MEMO OrgML is defined using MML. The four levels of abstraction defined by the MOF and the four levels of abstraction defined by MEMO are exemplified in Fig. 3.

The right of Fig. 3 exemplifies how a subprocess (i.e., BP activity) can be represented (with sample relevant attributes) at each level of the hierarchy. The rest of the paper shows how this classification hierarchy is used and enhanced to support the integration between types and instances, and to foster runtime models of BPs in a single tool. Beforehand, we discuss what actually characterizes ‘models@runtime’.

2.3 Models@Runtime

As discussed above, our aim is to extend the capabilities of BP models to support all phases of the BP lifecycle. The extended BP models can be updated at runtime, without having to regenerate or re-implement code of the BPM system (i.e., the runtime environment in which BP instances are executed and managed), they can be used to visualize BP instances during and after their execution, and they can be used to visualize key performance indicators (KPI) to facilitate online

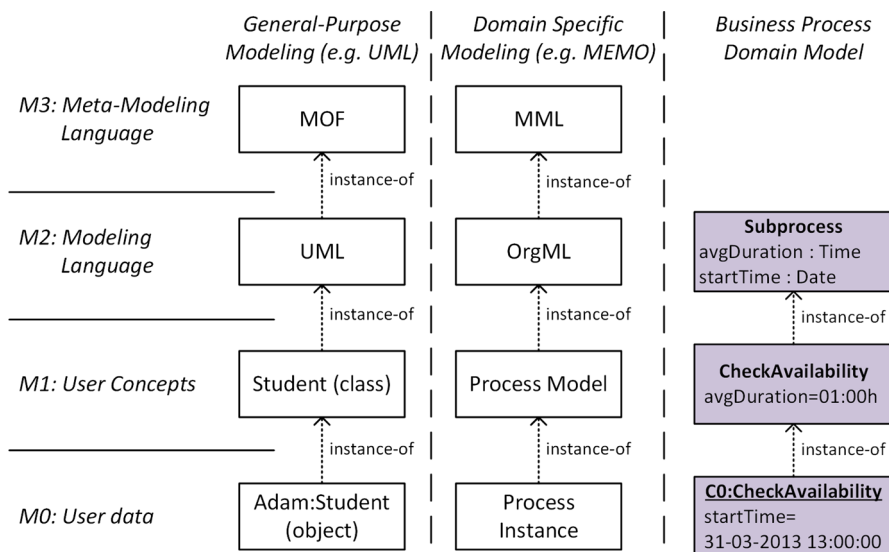


Fig. 3 The four-layer-hierarchy of modeling languages according to MOF and MEMO

analysis, monitoring, and controlling of BPs. Such capabilities, in fact, turn our BP models into what is known as runtime models or models@runtime. A runtime model is a conceptual model that is a “causally connected self-representation of a system” (Blair et al. 2009). For a model to be causally connected, the association between model and system needs to satisfy two criteria. Firstly, the runtime model has to always represent the correct system state, i.e., reflect up-to-date values of the objects managed in the runtime environment. Secondly, a modification of the runtime model has to provoke a correct system change (Song et al. 2011), i.e., the model can be used for manipulating the runtime system and the objects it manages. However, in the context of BP runtime models, the second criteria should be defined more accurately as, for example, we do not want changes in BP type models to affect the structure of previously executed BP instances that were valid at the time of execution. On the other hand, we are interested that changes of a BP model would be reflected in the runtime environment (which specifies how BPs should be executed) and consequently in new executions of the BP.

Lehmann et al. (2011) extend the above definition and argue that runtime models should include a prescriptive part—describing the system, in our case the BP enactment environment, and a descriptive part—describing the system state at runtime, in our case specific data of BP instances. Correspondingly, they differentiate between two types of model modifications that should be supported at runtime: modifications of the prescriptive part, in our case this would refer to the ability to modify the runtime environment code at runtime, and modifications to the descriptive part, in our case this would mean the ability to use the model for modifying runtime data of particular BP instances at runtime. Since most modifications of BP instance data should be external, e.g., caused by knowledge workers or software services, such modifications are less relevant in our case. Nevertheless, descriptive model modifications of BP instances would be required when the type model changes during its execution, i.e., when there are instances that are being executed and cannot be completed as originally designed. This type of modifications is, however, out of the scope of this paper.

Based on these characteristics of models@runtime we conclude that by turning a BP model into a runtime model, our objective of a BPM system supporting adaptations of the runtime environment and execution monitoring with BP models can be achieved.

More specifically, we consider two types of BP runtime models: (1) an instance model, which represents a particular execution of a BP (see for example the lower part of Fig. 2) and reflects the descriptive part (Meersman and Tari 2006); (2) a BP type model that defines the principal control flow for all its instances. To support business process management, the BP type model is enhanced with properties reflecting statistics and aggregations of runtime values of a set of BP executions (i.e., the descriptive part). For example, it can include performance indicators such as average activity cost, activity total costs per day, average activity duration, and event probabilities (e.g., the upper part of Fig. 2). In addition to that, in order to support ‘drill-down’ capabilities and foster understanding, it should be possible to navigate between the BP type model and its corresponding

runtime instance models. This requires integration of the type model and the actual executed BP instances captured in instance models.

3 Related work

In this section, we review work on the following related topics: support for BP changes at runtime and workflow flexibility, synchronizing build-time and runtime environments using runtime models, and process monitoring.

Flexibility and change in BPMS has been addressed in research since the mid-nineties when Workflow Management Systems (WfMS) gained higher popularity in the industry. (Schonenberg et al. 2008) distinguish between four types of flexibility: flexibility by design, flexibility by deviation, flexibility by change and flexibility by underspecification.

With respect to our objective to bridge the separation between build-time and runtime, the latter flexibility is the most relevant, as it requires applying changes of process specifications to the runtime environment and executed instances. Flexibility by underspecification is aimed at situations where process definition at design-time is incomplete and some decisions are left for runtime. Then, at runtime, predefined sub-processes (late binding) or not-yet defined sub-processes (late modeling) are selected to complete the process definition (Reichert and Dadam 1998). Flexibility by change, that is, the ability to modify a process definition at runtime such that all currently executing instances and also instances that will execute in the future are migrated to the new process definition, is partially relevant to us, as our goal is that changes to the BP type model will automatically affect the creation of new instances. However, changing currently executing instances is out of the scope of this paper. It should be noted that flexibility by design is inherently supported by BP modeling language using branching.

Works that address flexibility by underspecification and flexibility by change, as exemplified below, are aimed at different goals. First, bridging the separation between the modeling environment and the enactment environment (the WfMS), i.e., enabling BP model changes to seep into the enactment environment, so that new instances of the BP correspond to the modified model. Second, enabling dynamic changes of BP types in the WfMS at runtime (regardless to the modeling environment). Third, enabling dynamic changes of BP instances in the WfMS at runtime.

As stated in the introduction, our research is aimed at developing an integrated BPM environment that eliminates the separation of the design, runtime, monitoring, and evaluation phases. Hence, this paper is targeted at the first goal (and inherently supports the second goal as well).

A state of the art approach for bridging between the modeling and enactment (runtime) environments is to use XML file formats such as XML Process Definition Language—XPDL (WfMC 2012) or Web Services Business Process Execution Language—WS-BPEL (OASIS 2007) to interchange process models between the modeling environment and the enactment environment (i.e., the workflow engine). BP modeling tools provide automatic transformation of BPMN representation into XPDL and WS-BPEL formats. The resulting XML format is provided to the

workflow engine and guides the execution of the BP. While facilitating the ability to respond to change, this approach does not eliminate the separation of the two environments: whenever the BP model is modified it has to be regenerated and provided again to the workflow engine to keep the two environments synchronized. In addition, some modifications may require re-implementing code of the enactment environment.

Another approach that is aimed at bridging modeling and enactment is Worklets (Adams et al. 2006). This approach is based on Service Oriented Architecture (SOA) for providing flexibility by underspecification. A Worklet is defined as “a small, self-contained, complete workflow process which handles one specific task (action) in a larger, composite process” (Briand and Williams 2005). The approach allows for contextually selecting Worklets from a Worklet repository at runtime and incorporating them within a composite process model in places where the specification is incomplete. The Worklet approach is implemented as a service for the YAWL environment (Van Der Aalst 2005).

Capra and Cazzola (2012) propose an approach more similar to how this paper intends to deal with the challenges of bridging between the modeling and enactment environments. A formal process specification called “reflective Petri nets” is described. The authors use Petri nets to model a workflow for a given BP specification. For this Petri net, a metamodel is defined. Based on the meta-level information, the type level is manipulated at runtime using reflection techniques (an idea similar to meta-programming). However, in contrast to our proposed solution, this approach is not targeted at business users, as it uses Petri nets as the process modeling language.

To address flexibility by change, Reichert and Dadam (1998) present ADEPTflex, a workflow modeling language which uses directed, structured graphs to represent workflow specifications and instances. In order to enable dynamic changes to workflow types and instances at runtime, this modeling language is amended with a set of change operations, e.g., the insertion of a new activity that can be applied at runtime. The authors describe an algorithm for carrying out these operations based on graph transformations while respecting a set of defined consistency criteria.

In more recent publications, different strategies for synchronizing process instances with changes of the process workflow schema are put forth (e.g., Weske 2001; Ellis et al. 1995; Qiu and Wong 2007). Most of these strategies concentrate on validating change correctness with respect to the state of running workflow instances. In other words, they develop mechanisms for ensuring that process instances that are being executed are not damaged by the change and that they continue functioning as expected after the change. While this is an important aspect of workflow flexibility, it is not the focus of the work presented in this paper.

Abstracting from the peculiarities of BPM systems, the ability to dynamically change software structures at runtime is one of the driving motivations for the emerging research area of models@runtime (see Sect. 2.3). Research on models@runtime predominantly deals with software architecture (Bencomo 2009). For instance, Garlan and Schmerl (2004) and Georgas et al. (2009) propose frameworks for an architectural runtime configuration that leverage models of a running system’s architecture to enable adaptive systems as well as to facilitate system visibility

and management. Song et al. (2011) and Amoui et al. (2012) address the crucial challenge of synchronizing model and running system while changes on both sides occur. They offer model and graph transformations in combination with algorithms as solutions.

So far, there is almost no work on BP models@runtime. The only work we are familiar with targeting process models@runtime is presented in Sanchez et al. (2008), where executable models of workflows and a process modeling language are addressed. Here, dynamic adaptation of workflows is mentioned as a valid prospective of BP models at runtime.

Moving from the phase of process enactment to the phase of analyzing process execution data, the field of process mining (van der Aalst 2011) has produced substantial contributions to process evaluation and decision support. However, in contrast to the BP execution analysis outlined in this paper, process mining mainly applies data mining techniques to previously collected data sets of process instances recorded in event logs for retrospectively discovering and reconstructing process type models, for identifying deviations from process designs and sometimes even for predicting (online) the behavior of cases that have not yet completed. Analyzing such log files, usually represented in text format, requires some effort of first parsing and converting log data into a formal data structure (van der Aalst 2011). Event log files are also used for monitoring policy-compliance and usage control (Ly et al. 2015). For example, Basin et al. (2010) present a monitor that determines whether recorded system events comply with security policies that are formalized using first-order temporal logic. Hilty et al. (2007) also use temporal logic to monitor usage control at runtime. As opposed to these event log based approaches, our approach uses the BP models themselves for monitoring and evaluation of BP executions. Therefore, it helps to avoid the tool fragmentation mentioned above. Nevertheless, it is possible to apply such policy compliance and access control monitoring approaches on BP runtime models as well—such monitor functionality could be implemented as part of the OrgML metamodel entities and applied at runtime when processes are executed.

4 Requirements and challenges for the targeted BPM environment

In light of the BP lifecycle fragmentation discussed in the introduction and the presented related work, this section defines requirements that a proposed solution for continuous BPM should satisfy.

BPM practitioners commonly indicate that BPM tools are relatively fragmented and that “activities completed in one phase with one type of tool do not translate to the next steps of the lifecycle” (Bandara et al. 2007). In particular, the separation between process design and process execution is stressed as problematic, as there is no “tool to design process directly into runtime” (Bandara et al. 2007). This separation especially impairs implementation of BP design changes. Adapting the runtime system to the new structure of BPs requires a lot of rework and time (Bandara et al. 2007; Capra and Cazzola 2012; Qiu and Wong 2007; Weske 2012).

Therefore, the targeted BPM environment should overcome this separation by consistently supporting all phases of the BPM lifecycle, i.e., allowing for a transparent transition between them. In particular, we derive that the following general requirements (*GR*) should be satisfied:

- GR1 It should be possible to directly design BPs into a runtime system. That is, there should be a transparent transition from the design phase to the execution phase, without a need to generate or implement additional code. As a result, BPs can be immediately enacted once the design of the BP is completed. *Rationale:* Managers need to know how each BP type is defined to ensure the proper course of work. Yet, in many cases there is a substantial difference between the abstract process models specified at process design phase and their implementation, which leads to “a large amount of rework and sometimes loss of information in the process of translation” (Bandara et al. 2007). Consider for example a situation in which a BP defines that a customer’s order is accepted only after the customers’ credibility had been checked, however, the BPMS defines that the order is accepted without any check. Even in BPM suites that support both the design and enactment of BPs such that the enactment is based on the BP designs (e.g., YAWL, Bizagi, and jBPM), the transition from design to enactment requires generation of code (e.g., XPDL) leading to an additional representation of the BP (the workflow schema) to be maintained and deployed to the WfMS. Thus, the proposed solution should not only enforce compliance of BP design and execution, but also a direct/transparent transition between them.
- GR2 The BPMS should easily adapt to changes in BP designs without requiring regeneration, reimplementation or recompilation of the system. *Rationale:* Managers should be able to adapt a BP type to changing requirements. Following the previous example, consider that it should be added to the BP that there is no need for a credibility check if the total sum of an order is lower than a certain value. Whenever such changes to BP designs occur, regeneration or reimplementation of BPMS is required. This consumes extensive effort and time, and may lead to situations where the designs are not in sync with the runtime environment.
- GR3 It should be possible to monitor the execution of BPs (e.g., current state of execution, completed tasks, involved employees, and IT resources) and to analyze their performance (e.g., durations and costs of different activities and probabilities of different events) at runtime. The transition from the execution phase to the analysis and evaluation phase should be automatic and transparent. Executed BPs should be immediately ready for monitoring and analysis, that is, no special preparations and pre-processing, such as data extraction, cleansing, or data type conversions of executions history should be required in order to analyze the executions. *Rationale:* managers need to know characteristics of particular process instances and of sets of process instances. For example, when a particular process started and terminated; what the average execution time of all processes of a certain type in 2016 was; or what the response value of a web-service call was and how many

of the calls returned an error. In most suites, the transition from the execution phase to the evaluation phase requires investment of time and effort. In addition, working on different tools in these phases (as is the case in many organizations) may again lead to loss of information and miscommunication between responsible stakeholders.

- GR4 Similar visualizations should be used for representing BPs in the different phases of the BPM lifecycle, in particular, in the design, monitoring, and evaluation of BPs. *Rationale:* Using similar visualization for different tasks would foster communication between the different stakeholders performing them. Furthermore, it has been suggested that visualizations of process instances employed in BPM systems is insufficient for process managers (Bandara et al. 2007). Hence, BPM would benefit from consistently using visual renderings of BPs. Since the design phase visualizations are primarily intended for business users, they are assumed to be best tailored to process analysts as well.

In order to foster integration of all phases it is proposed to enhance BP models with runtime model capabilities (see Sects. 1 and 2.3), so that they can serve as a comprehensive tool for supporting BPM activities throughout the lifecycle phases. In the following, we define concrete requirements (CR) that should be addressed by such enhanced BP runtime models. Satisfying the concrete requirements would guarantee that the general requirements are also satisfied.

- CR1 *Direct enactment:* BP type models should directly enable the creation of BP instances. The transition between the design of a BP via a type model and the creation of corresponding instances (i.e., BP enactment) should not require any additional transformation or implementation of code. *Rationale:* This concrete requirement addresses GR1 by ensuring a transparent and direct transition from design to execution, that on the one hand ensures design-execution compliance and on the other hand ensures an effortless transition between them.
- CR2 *Synchronized representations of design-time and runtime BPs:* On the one hand, changes to BP type models should be automatically reflected by the runtime environment and as a result affect the execution of new instances. On the other hand, previous executions should not be affected, as we would not want past execution data to change. In addition, when the design of a BP instance changes during its execution, it should be possible to determine whether the not-yet-executed activities of the BP should follow the new design or follow the original design. *Rationale:* this concrete requirement addresses GR2 by ensuring that changes of BP types can be automatically rendered to the runtime environment and thus to the instance level.
- CR3 *Representation of BP instance state in real time:* In order for BP instance models to represent particular executions and reflect the correct state of BP instances, they should capture abstractions of BP instances as they progress. *Rationale:* this concrete requirement addresses GR3 by providing managers

information on event and activities characteristics of particular process instances. Van der Aalst (2011) identifies that essential attributes reflecting event data in event logs comprise event timestamps, activity cost, participating resources (e.g., employees and IT components), and source/target activities of events. This is an essential set of attributes that should be included in instance models to describe currently executing or executed BP instances (see ③, ④, and ⑤ in Fig. 2). Of course, additional attributes could be included as well.

- CR4 *Representation of BP type state (performance indicators) in real time:* BP type models should be enriched with performance indicators that aggregate information and calculate statistics on BP executions as they progress. For example, a BP activity should include attributes such as average duration and average cost (see ① and ② in Fig. 2). *Rationale:* this concrete requirement addresses GR3, by providing managers aggregative information on sets of instances, which allows them to analyze KPIs of BP types at runtime.
- CR5 *Data synchronization of type and instance levels:* Performance indicators represented in BP type models should be updated as executions of the BP progress. *Rationale:* this concrete requirement addresses GR3 by ensuring that when changes occur at the instance level, they will correspondingly affect the type level. For example, when a particular activity of a BP instance is completed the average duration indicator of that activity type should be updated according to the duration of the particular activity, allowing the manager to see up-to-date BP KPIs. This means that a BP model and the information of its executed instances should be synchronized. To support such synchronization, BP types should be integrated with and aware of their instances at runtime
- CR6 *Navigation between a type model and its corresponding instance models:* it should be possible to navigate from a BP type model to its executions, captured in the respective instance models. *Rationale:* To foster analysis of problematic performance indicators of a BP type, it should be possible to drill-down to the source of the problem, i.e., to search for problematic instances (e.g., ⑥ in Fig. 2).
- CR7 *Similar visualizations of BPs throughout the BPM lifecycle:* the same BP model visualizations should be used for design, monitoring and evaluation tasks. Thus, BP instance models and BP type models should have similar representation. *Rationale:* this concrete requirement addresses GR4 by reusing the representation of the design-time BP type models for BP monitoring and evaluation tasks, in order to facilitate communication between the different stakeholders performing them. For example, this would allow managers to easily compare particular instances of a BP to the way it was designed and identify unusual behavior. Additionally, this would provide managers a comprehensive quick impression of the status of the various activities in the process.

Implementing a BPM environment that addresses these concrete requirements is a challenging task. In fact, in the next subsections we argue that this objective cannot

be addressed using “ordinary” modeling tools and traditional object-oriented (OO) programming languages. We identify two main challenges with respect to the shortcomings of traditional modeling tools and programming languages.

4.1 Challenge 1: including runtime abstractions in the modeling language

To support managers’ need to know characteristics of particular process instances and of sets of process instances (e.g., when a particular process started and terminated and what the average execution time of all process instances of a certain BP type last year was), runtime abstractions (as identified in requirements *CR3* and *CR4*) need to be included in the BP modeling language (for example, the modeling language should include concepts like start time, termination time, and average duration). Ignoring these runtime abstractions in the language (M2) implies burdening language users with the responsibility of defining type level runtime attributes and associations, such as a timestamp and cost, in every BP model they create (Atkinson and Kühne 2001). This hampers a shared understanding and reusability of resulting models, as an apparent abstraction is ignored, resulting in dangerous redundancies (Frank 2011b) such as multiple definitions of the same attribute in different process types.

Nevertheless, popular modeling frameworks such as the Eclipse Modeling Framework (EMF) do not enable capturing instance level attributes already within the modeling language classification. To overcome this, we use the MEMO notion of an intrinsic feature (Frank 2012), as discussed in Sect. 5.3.

4.2 Challenge 2: the edit-generate-compile-validate cycle

Satisfaction of *CR1* and *CR5–CR6* requires that type and instance models be integrated in adaptable runtime objects of the modeling environment. Existing modeling editors based on the state-of-art approach, in which modeling editors are generated from a metamodel of a modeling language, cannot be feasibly used to achieve such integration of type models, instance models, and runtime BP instance objects. The reason for that is that most meta-modeling facilities (e.g., EMF), are based on mainstream OO languages, which typically support two language levels: classes and objects. As a result, type models created in existing modeling editors are represented by M0 objects, which cannot be re-instantiated to represent instances of the desired type models (Atkinson and Kühne 2001). The state-of-the-art modeling environments resolve this difficulty by using an “edit-generate-compile-validate” cycle to first generate classes representing the modeling editor software for creating type models and second to generate instance level editors (based on the type models) from which instances can be created (see Fig. 4). However, runtime synchronization between type and instance level is impeded as types and instances are depicted in disconnected editors, as is synchronization of the evolution of type level with instance level, which requires an integration of the cycle whenever a type model changes. Moreover, relevant type model editors (e.g., performance indicators) must immediately reflect the necessary changes when a BP instance is created or changed

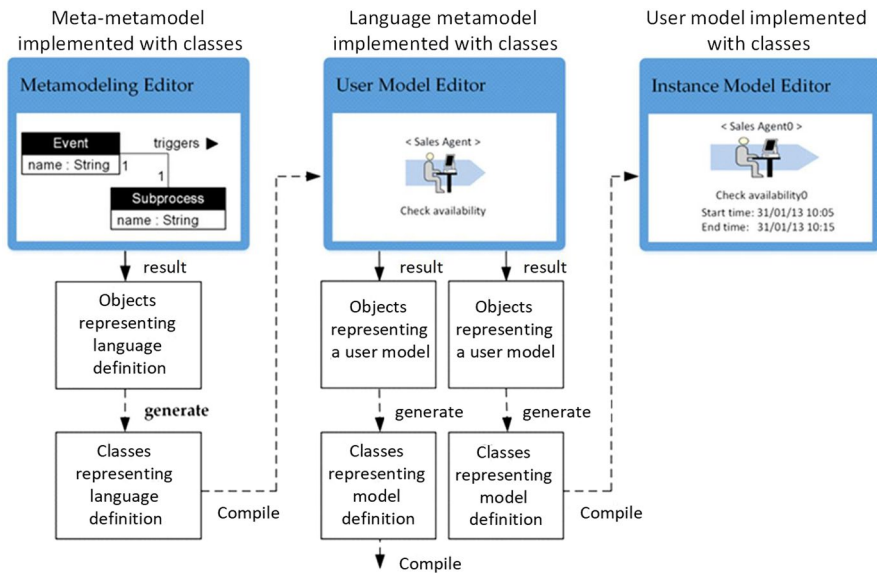


Fig. 4 Edit-generate-compile-validate cycle for meta-modeling based model editor creation

(e.g., upon termination of a subprocess). Consequently, recompilation of the editor, which is a necessary element of the OO approach, is not feasible for runtime models and does not provide the required flexibility (Lehmann et al. 2011)[1].

One possible compromise that circumvents these obstacles involves use of a custom relationship to mimic the classification relationship of types and instances, for example by introducing separate classes that represent type features and instance features and associating them using a “typeOf” relationship. However, this solution does not represent a genuine “instance of” relationship between class and object. The additional code required in this case constitutes a severe threat to system integrity (Kühne and Schreiber 2007). In our approach, we substitute XMF, a meta-programming language, for Java, a traditional OO language, XMF is further described in Sect. 5.1.

5 Developing a modeling environment for BP runtime models

In the previous sections, we proposed extending BP models with runtime capabilities, making them suitable for supporting the entire BPM lifecycle, consistently connecting all of its four phases. In Sect. 4, we identified several requirements and challenges that a corresponding BPM environment should address. The first challenge relates to the ability to include runtime abstractions of BPs within BP models. The second challenge relates to the ability to integrate and synchronize BP runtime models on different levels of classification and the actual BP instances represented in the BPM system. To address these challenges, our solution implements OrgML of the MEMO framework using XMF. In the following subsection we present XMF and

discuss how it is used for the creation of BP runtime models that handle the identified challenges. We demonstrate our solution through a prototype.

5.1 XMF

A meta-programming language is distinguished from regular programming to modify language concepts. Thereby it is possible to specify and implement new languages as instances of the metamodels that form the foundation of a meta-modeling language (Spinellis 2008). The meta-programming capabilities used in the presented work derive from XMF (eXecutable Metamodeling Facility) (Clark et al. 2008).

XMF is a language execution engine featuring a metamodel called XCore (Clark et al. 2008). Every language that is specified in XCore can be executed by XMF. XCore follows the golden-braid architecture (Hofstadter 1980), which emphasizes the fact that metamodels, models, and instances are all relative concepts based on the fundamental property of instantiation. A pivotal characteristic of the golden braid architecture is that every class is an instance of a (meta) class and that every class is an object with state and behavior at the same time. This is achieved by the recursive structure of XCore: every class is an instance of *Class* and *Class* itself inherits from *Object* (see Clark et al. 2008). As a consequence, in XMF every class can be executed and manipulated during execution.

Because of this architecture XMF enables access to and modification of its own specification and its runtime system. Therefore, there is no clear distinction between the language and the corresponding meta-language, meaning that XMF is reflective: the runtime system allows to inspect not only the state of objects on M0, but of every class on any classification level. Additionally, XMF provides tools for developing and compiling new languages. This makes XMF a meta-programming facility that allows for the execution of code written in different languages in the same runtime system (cf. Frank 2014).

XCore is complemented with an executable version of the Object Constraint Language (OCL), called XOCL. XOCL includes action primitives for expressing constraints, actions, sequential execution paths, and imperative control structures.

The described architecture of XMF, in which instances of classes can again serve as classes that can be instantiated, justifies why implementing a BP modeling tool with XMF avoids the “edit-generate-compile-validate”-cycle (specified in Sect. 4). For example, BP modeling language concepts can be instantiated to form a BP type model (e.g., order processing) that can be again instantiated to represent particular BP executions—such that all entities are executed in the same runtime environment. In addition, since every class is an object, classes are provided with executable behavior. For example, classes composing a BP type model are able to access their instances in order to obtain concrete execution data. XMF is more powerful than other OO languages that are based on a golden braid architecture, for example Smalltalk. While supporting meta-classes, which enable accessing the class level (M1) at runtime, Smalltalk does not allow for the definition of meta-classes above M2 and only allows a class (M1) to have a single meta-class, whereas in XMF there is no limit to the number of abstraction levels and a class can have more than

one meta-class, which as can be seen in the following section, is essential for our solution.

5.2 Tool implementation architecture

The recursive language architecture featured by XMF is exploited by the proposed modeling tool for coping with the challenge of dynamically changing objects at both type and instance levels, as implied by BP runtime models. This is accomplished by a pivotal idea of cross-level inheritance: concepts at each level of the tool's architecture inherit from XMF Concepts, as described in the following paragraphs.

As a first step to implement a modeling editor featuring OrgML, we reconstruct the MEMO MML (see Sect. 2.1) with XMF meta-modeling concepts (XCore). In other words, we instantiate XCore concepts like 'Class' and 'Attribute' in order to model the MML at M3, as shown by the two upper levels of Fig. 5. Because of this instantiation (dashed arrows in Fig. 5), MML concepts are objects and thus can

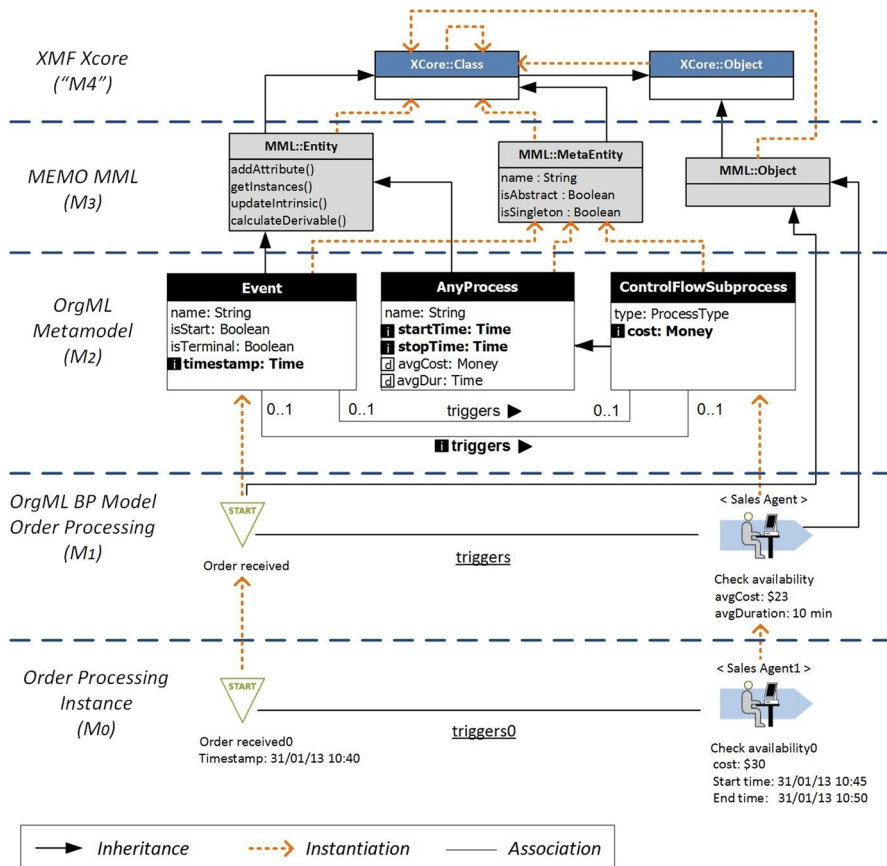


Fig. 5 MEMO language architecture for business process runtime models

be executed and manipulated, as discussed in Sect. 5.1. In addition, MML concepts are set to inherit XMF concepts (full arrows in Fig. 5) in order to add executable semantics to instances of the MML. Through this inheritance, instances at M2 are equipped with features like setters and getters, which are crucial to modeling editors. From the XMF implementation of MEMO MML, a modeling editor for MEMO MML is generated (XMF offers built-in support for modeling tool generation).

Secondly, we create the OrgML metamodel with the XMF-based MEMO MML editor and account for the runtime model capabilities discussed in Sect. 4. For this purpose, the outlined runtime functionality relevant to all instances of ‘MetaEntity’ (the meta-meta-class all concepts of OrgML are instantiated from, see middle level of Fig. 5) is captured in a concept called ‘Entity’. The functionality mainly comprises navigation from type to instance models, calculation of performance indicators, and instantiation of intrinsic features (described below in Sect. 5.3). ‘Entity’ also inherits from XMF’s ‘Class’ in order to add execution semantics to instances of MetaEntity and enable their further instantiation. Thereby, every OrgML concept (e.g., ControlFlowSubprocess and Event) transitively through ‘Entity’ inherits from XMF’s ‘Class’, providing its instances (e.g., particular event types and activity types) with instantiation capability and with the described ‘Entity’ functionalities. In combination, we can use the implementation of OrgML to create an OrgML editor at runtime with XMF. BP type models created with this editor (M1 level) can be directly instantiated into BP instance objects (M0 level), i.e., used for executing a workflow. Furthermore, they can be used for creating instance model editors at runtime, in which the progress of BP instances can be monitored.

It should be noted that, while languages such as BPMN and YAWL are sometimes referred as “executable”, the models created using these languages are not literally executable, rather, being represented by objects on M0, they can only be used for generating a workflow schema (e.g., XPD L code) that can be then executed in a WfMS (YAWL even comes with its own WfMS). As opposed to that, BP type models created in the proposed OrgML editor are literally executable. I.e., on the one hand, created type models (M1) are composed of classes, which are also objects that can be executed and manipulated; On the other hand, these classes can be instantiated to create objects which compose the instance model. Furthermore, BP type classes/objects can execute methods (inherited from ‘Entity’) for accessing and manipulating their BP instances objects.

The fact that all BP model objects at M1 (i.e., its activity types, event types, and flow types) can be instantiated into M0 objects implies that changes to the M1 objects are immediately reflected in new instances at M0, that is, instances of a BP are inherently in sync with the evolution of the BP type.

The inheritance hierarchy enabling the instantiation of dynamic type level entities (i.e., entities that can be modified at runtime) is given at the bottom of Fig. 5. In fact, all levels (M1-M3) can be dynamically manipulated and affect the levels below them at runtime.

‘Entity’ also encapsulates an operation for querying instances of a type—getInstances(). With said operation, M1 level concepts are able to get a list of their instances, whether running or terminated, which serves as a foundation for integrating and navigating to the instance level at runtime.

To summarize, Fig. 5 exhibits a pivotal idea of the proposed multi-level modeling tool: cross-level inheritance. Indeed, ‘Entity’ inherits from ‘Class’, while being at the same time an instance of ‘Class’; all OrgML entities specialize ‘Entity’, while being at the same time instances of ‘MetaEntity’. Note however, that ‘MetaEntity’ and ‘Entity’ are at the same level of abstraction. Because of the (transitive) cross-level inheritance of XMF concepts within the presented architecture, the role XMF plays in relation to the four modeling levels of MEMO is twofold: on the one hand, XMF can be considered a fifth level of abstraction above M3; on the other hand, XMF serves as a modeling infrastructure offered to all MEMO modeling levels via transitive inheritance. Thus, it can be said that all objects in XMF are level-agnostic. Furthermore, the proposed multi-level modeling architecture enables adding more levels of abstraction. This could be used for introducing additional domain-specific concepts such as domain specific activity and event (meta) types (e.g., “check availability” activity (meta) type that would be then instantiated into more specific types), which would foster reuse and integrity.

It should be noted that for readability purposes, every level in Fig. 5 contains only an exemplary set of concepts.

5.3 Including business process runtime abstractions

In Sect. 4 we argued that OrgML should be enhanced with abstractions of runtime properties such as event timestamp and subprocess start time. In addition, it should be enhanced with performance indicators that reflect statistical operations on runtime data of instances.

While popular modeling frameworks such as the EMF do not enable capturing instance level attributes already in the modeling language classification, MEMO MML allows for such definitions. This is enabled by the notion of intrinsic features (Frank 2011b), which is similar to ‘Powertypes’ (Odell 1994) and ‘Clabjects’ (Atkinson and Kühne 2008). Features such as meta-entities, meta-attributes or meta-associations of the metamodel (M2) can be defined as intrinsic, meaning that they are relevant only at the instance level (M0), while at the type level we abstract from them. Intrinsic features are visually marked with a boxed letter “i”. Hence, adding specific intrinsic attributes to OrgML makes it capable of reflecting instance-level data, i.e., instance models. For example, defining a meta-attribute ‘cost’ for the OrgML concept of ‘Subprocess’ and marking it as intrinsic entails that the attribute—in contrast to regular meta-attributes—is not instantiated into a particular value at the M1 level, but at the M0 level (see illustration in Fig. 5). This is implemented by copying the meta-attributes that are marked as intrinsic to the M1 level as regular type-attributes, which can be then instantiated at the M0 level. In the same way, defining the meta-entity ‘RoleAllocator’ as intrinsic entails that this entity is not instantiated into particular entities on the M1 level but only on the M0 level to capture particular RoleAllocator instances (e.g., particular employees).

Figure 6 describes a simplified metamodel of OrgML, in which some examples of runtime abstractions are highlighted: start_{Time}, end_{Time}, and state of AnyProcess; timestamp and id of Event; cost and laborCost of ControlFlowSubProcess;

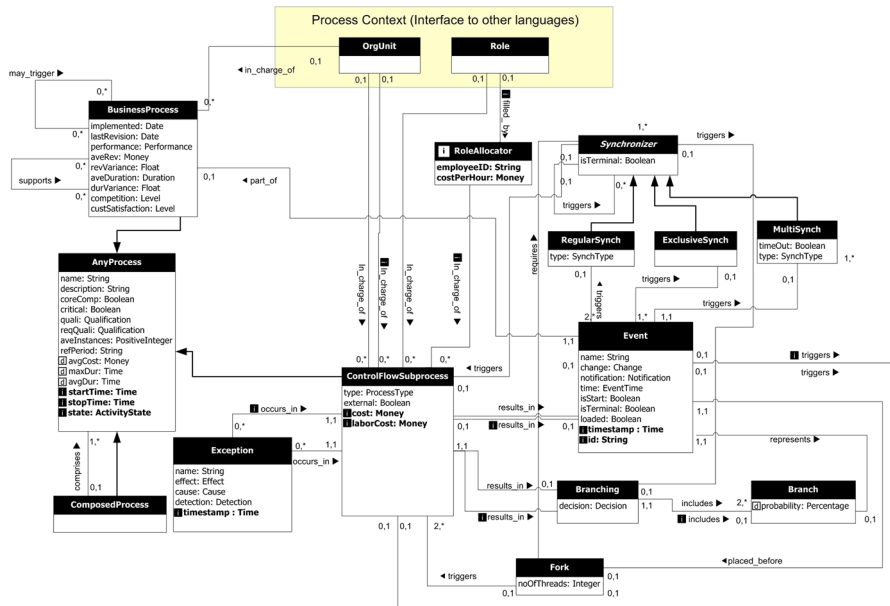


Fig. 6 Simplified metamodel of OrgML enhanced with runtime abstractions

timestamp of Exception; and the entity RoleAllocator. While not shown in Fig. 6, OrgML is also integrated with other MEMO DSMLs, for example—ITML (see Sect. 2.1), allowing to relate BP activities with IT resources that are used for performing them and accordingly associating activity instances with related IT resource instances. This supports analyzing, for example, failing machines or software services that interrupt the execution of BP activities.

Additional specifications of relations are required at M0. The problem is that until BP type models are defined at M1, we do not know which relations are valid at M0 (i.e., the source and target entities of each relation are not determined yet) and what are their cardinalities. Consider for example, the meta-association ‘triggers’ from Event to ControlFlowSubprocess in Fig. 5. Only after a specific ‘triggers’ association between ‘Ordered received’ and ‘Check availability’ is defined by the BP modeler (this happens at design time) can we define that only instances of ‘Ordered received’ and ‘Check availability’ may be related at M0 by a triggers instance. In addition, the specified cardinality of this meta-association is relevant for associations that are instantiated at M1 but not for M0 instances.

To address this, we use a two-step solution. The first step is to define for each meta-association in OrgML a corresponding intrinsic association that would specify the relation at M0. For example, in addition to the ‘triggers’ association an intrinsic ‘triggers’ association is defined. The second step takes place in the modeling environment once the definition of the BP type model at M1 is completed, when at the request of the modeler, an operation called ‘updateIntrinsics()’ is executed. This operation sets the sources and targets of all the intrinsic association types based on the associations that are defined in BP type model. Thus, for example based on the

association ‘triggers’ from ‘Order received’ to ‘Check availability’, defined at M1, it sets the intrinsic association ‘triggers’ that starts from ‘Order received’ to be ‘Check availability’.

The implementation of performance indicators starts with modeling them as meta-attributes in the OrgML definition. These meta-attributes are marked ‘derivable’ to indicate that they require a calculation (e.g., see meta-attributes marked with a ‘d’ icon in Fig. 6). To provide the calculation logic, we specify a generic operation for ‘Entity’ (‘calculateDerivable()’ at M3 in Fig. 5), which offers an interface for setting a data source on which the calculation is performed and a mathematical operation. Additionally, we add a set of generic calculation operations to ‘Entity’, e.g., mean and maximum value. However, the specific derivation often needs to be defined by the language designer. Therefore, custom derivation operations can be added to any child class of ‘Entity’. For instance, in order to calculate the average duration of a subprocess, we add a calculation operation to ‘AnyProcess’ which is able to identify the amount of time incurred between its `startTime` and `stopTime` of a particular process (or subprocess) type. With these generic derivation operations, formal connections between a type and its instances are created at M3. Thus, a performance indicator (e.g., an activity’s average cost) is constantly calculated from the spot-on runtime information of the instance level (e.g., activity’s cost), regardless of the BP type model (M1) created by the user.

Finally, in order to automatically synchronize representations of BP type and instance models with the actual runtime information of BP instances (i.e., synchronize derived attributes of the type level with intrinsic attributes of the instance level), our prototype uses change listeners, called daemons in XMF. With daemons, we exploit the fact that in XMF every element is aware of the higher-level concept it is an instance of. Furthermore, in our OrgML implementation every ‘Entity’ instance is able to query for its instances. Correspondingly, daemons can notify types on changes of their instances. Thus, when the execution of a BP instance proceeds, corresponding listeners are notified and automatically update derivable attributes in the type model. For example, when an instance of a subprocess (M0) is finished, a listener notifies the subprocess type of this instance so that values like the average cost of said subprocess are recalculated. The update of the BP type model can take place at runtime, because, owing to the features of XMF, the ‘classes’ of BP instance models are stateful runtime objects at the same time.

When combining type level performance indicators and instance level attributes in integrated modeling editors, we arrive at a tool serving process design on the one side, and versatile process monitoring on the other, as demonstrated in Sect. 6.1.

6 Evaluation

The artifact developed in this research is a BP modeling method, which consists of a conceptual modeling language and a corresponding modeling environment that provides models@runtime features so that created BP models can be used for enactment and for evaluation of BPs. The evaluation of the artifact includes two parts. First, a validation of the developed prototype and demonstration of its capabilities

through various use cases (Sect. 6.1). Second, an evaluation of the proposed modeling method (Sect. 6.2). We start by providing background and motivation for our chosen evaluation procedures (requirement-based evaluation and expert-panel evaluation), and then present their application.

6.1 Prototype validation through a use case

At this point, the proposed BPM environment is not implemented in a ‘real-world’ setting, where it interacts with actual process-oriented systems or humans that initiate and manage BPs. Thus, in order to evaluate the prototypical modeling environment, we have implemented an agent that imitates external events which initiate the execution of BP activities based on predefined probabilities of the occurrences of alternative events (e.g., the probability that a product is available or not). As a result, the agent is able to initiate the instantiation of a given BP type model created with our prototypical editor to create various BP instances. Thus, this lets us validate that runtime models behave as they should, that is, visualizations of the created instances are correct and the BP type models present correct runtime data. We have validated our prototype against several business process models. Here we present the case of order processing.

A screenshot of an Order Processing type model that was created in the prototypical modeling environment is presented in Fig. 7. Before the BP was executed, the derivable attributes of average cost and average duration of each activity (subprocess) are empty (null).

Once the Order Processing model was created, we used the agent to complete several executions of this process, i.e., mimic a user/machine that initiates process instances. A screenshot of one of the instances, OrderProcess_M03, is given in Fig. 8. As can be seen, the instance model includes runtime information,

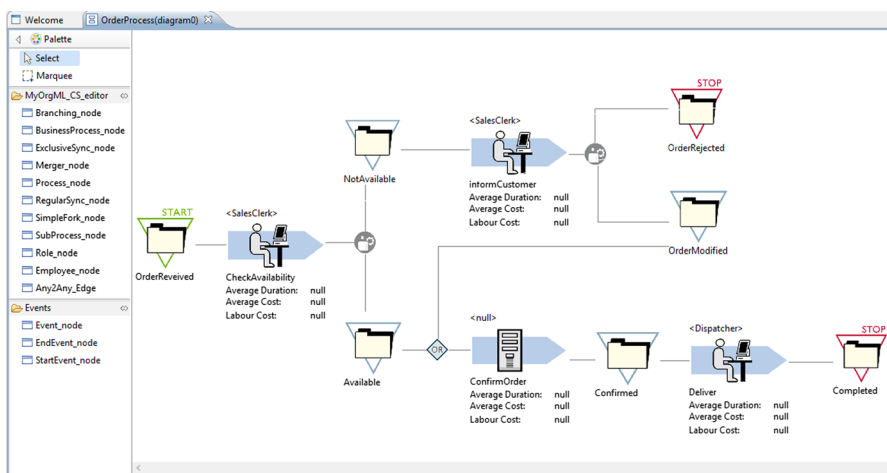


Fig. 7 A screenshot of an order processing type model created in the modeling environment

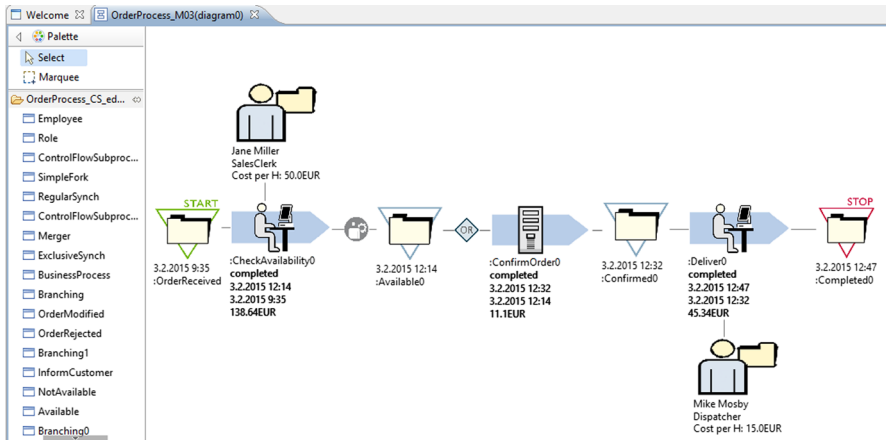


Fig. 8 A screenshot of a completed order processing instance model created in the modeling environment

such as the cost, start and stop times of activities, activity state, and participating resources (i.e., employees).

Figure 9 shows a screenshot of another instance of order processing, Order-Process_M04. This screenshot was taken while the instance was being executed, i.e., during the execution of the deliver activity.

It should be noted that the OR branching has no actual correspondence on the instance level, as a branch can either be processed or not in a process execution. Nevertheless, it is rendered in the M0 model to indicate that a branching instead of a regular sequence occurred at this point.

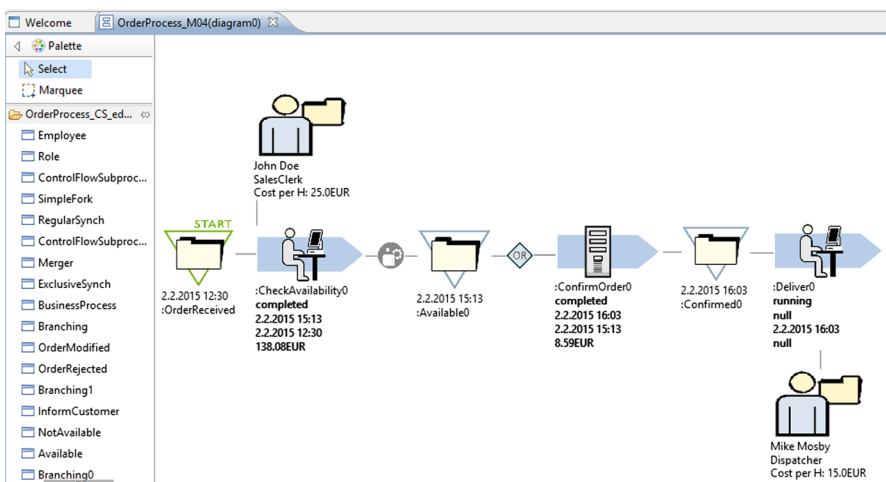


Fig. 9 A screenshot of a running order processing instance model created in the modeling environment

In addition, once instances are created, the model's derivable attributes are immediately updated based on its instances' values. This is demonstrated by the screenshot given in Fig. 10. The figure shows the average cost and average duration of each activity over ten generated instances.

This use case demonstrates that the prototype indeed integrates between the type model and its instances and shows that changes at the instance level are automatically reflected in the type model.

In the opposite direction, when the design of a BP type model changes, new instances of the BP immediately comply with the new design. This is in fact a fundamental feature of the proposed approach, as BP instances are literally instantiated from BP type models rather than just conceptually following their definitions, after being generated in a different language, as the case is for other approaches. For example, we added a new activity “*CheckCompleteness*”, in which the order completeness is checked, right before the “*CheckAvailability*” activity, and used the agent for creating new instances of the updated process. As a result, the new instances follow the structure of the updated process type, while the structure of instances of the previous design is not affected, as intended—we would not want historical information on previous executions to change. The derivable attributes on the other hand, continue to be updated as new instances are being executed.

6.2 Evaluation of the proposed method

As an artifact of this research, a BP modeling method was developed including a conceptual modeling language and an environment that contains models@runtime features, allowing the resulting BP models to be used to enact and evaluate BPs. However, research that aims to develop modeling methods as artifacts must be justified according to scientific standards (Frank 2006; Schelp and Winter 2006). The required justification is challenging as the successful use of such a modeling method depends on numerous factors related to users' qualifications, experience, learning

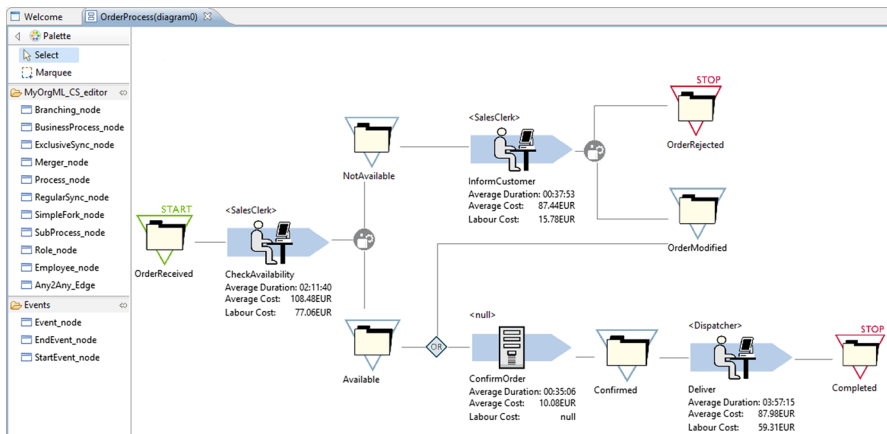


Fig. 10 A screenshot of an order processing type model right after it was executed

curve, and attitudes. Moreover, where such an artifact is novel, users may not have a clear understanding of current and future applications of a BP runtime model, and as a result may be unable to evaluate its practical benefits. Consequently, the success of a method cannot be comprehensively evaluated by empirical tests, because such an evaluation entails comparison of performance on success criteria against the factual world, omitting reference to other possible worlds (not yet accounted for) that would produce other evaluation results (Rorty 1999).

To address these concerns, our work is grounded in a justification approach (Frank 2006), which suggests two main guidelines for the research process: transparency of assumptions and multi-criteria justification (Frank 2006). Transparency requires identification of all non-trivial assumptions that are not obvious or deduced from established knowledge, including requirements, design decisions, and the evaluation of the method against the requirements. Multi-criteria justification reflects the belief that various approaches to substantiate an assumption are available, and approaches are selected based on what is considered to be the suitable theory of truth and feasibility of corresponding justification procedures (for example, justification through empirical testing corresponds to correspondence theory, discursive evaluation is appropriate for consensus theory, and evaluation of consistency with existing knowledge corresponds to coherence theory).

For a discursive evaluation to be satisfactory, it must meet several prerequisites in addition to consensus (Habermas 1984), including expertise, traceability, openness, absence of concealed political preferences, and absence of coercion or incentives that affect preferences of certain arguments. Although a consensus achieved in a rational discourse does not guarantee the ultimate truth, such consensus may offer the only chance of developing an evaluation or justification in cases where correspondence or coherence theory seem insufficient or inapplicable. Ideally, rational discourse is performed by multiple dedicated experts, however, it is also possible to conduct a virtual discourse, a relaxed form of discursive evaluation, that is based on carefully gathering and confronting arguments from different perspectives.

As mentioned above, empirical tests have not been included in the present study due to subject contingency and lack of feasibility. Instead, the justification procedures employed in the current research include two parts: the first—requirement-based evaluation—is a combination of a virtual discursive evaluation and an application of the coherence theory of truth (i.e., substantiation of assumptions by reference to a body of literature). The second part is a rational discourse evaluation using a panel of seven IT and BPM experts. We describe these justification procedures in the following subsections.

6.2.1 Requirements-based evaluation

Four general requirements were established for an integrated, continuous BPM environment that supports the entire BPM lifecycle and facilitates consistent and transparent transitions between its phases. The requirements were mainly derived from the existing literature and the analysis of existing BPM tools. We also elicited

feedback from BPM professionals and researchers to ensure consensus with respect to the requirements. As the proposed method is aimed at meeting these requirements with BP runtime models, we elaborated the general requirements into seven concrete requirements to be satisfied by such BP runtime models. Satisfying these seven concrete requirements would thus guarantee that the four general requirements are fulfilled. While the derived set of requirements may not be exhaustive, we deem them pivotal for constructing the required modeling method and corresponding BPM environment.

Next, the design adequacy is evaluated against the concrete requirements *CR1* to *CR6*.

According to requirement *CR1*, BP type models should support direct enactment of BP instances, so that the transition from design of a BP to its representation in the runtime system would be automatic and transparent. This requirement is supported by the XMF-based architecture, as the proposed architecture allows objects of a BP type model to be directly instantiated into BP instance objects (without requiring any code generation and recompilation). This is demonstrated by the use-case presented in Sect. 6.1, where concepts of the Order Processing BP type are instantiated to represent particular (simulated) executions of the BP. Satisfying *CR1* ensures that *GR1* is also satisfied.

Requirement *CR2* further extends *CR1* and requires that when the design of a BP changes, new executions of the BP would automatically conform to the new design. This is also satisfied by the ability of proposed BP runtime objects to be instantiated—since M1 type level objects can be directly instantiated into instance level (M0) objects, changes to the M1 objects immediately seep to new instances. Past instances created before the change, on the other hand, are not affected. Nevertheless, situations in which the design of a BP changes during its execution, where some activities, events, and flows follow the old design and others follow the new design, require special handling. In such situations and in general, the ability to manage different versions of BP runtime models and their instances requires developing a versioning mechanism for BP models. This mechanism is left for future research. Satisfying *CR2* ensures that *GR2* is satisfied.

Requirement *CR3* entails enriching BP instance models with runtime (instance-level) characteristics so that they can represent data of particular executions. As shown in Sect. 5.3 and demonstrated in Sect. 6.1 the proposed solution uses the mechanism of intrinsic features to capture runtime characteristics in the BP modeling language. Thus, created BP models include both type and instance level characteristics, which allow for using them both for representing type models and their instances.

Requirement *CR4* calls for extending BP models with performance indicators. Section 5.3 presents the mechanism of derivable attributes. With this mechanism a derived attribute is defined as the output of an operation performed over attribute values of corresponding instances. This mechanism satisfies requirement *CR4*, as demonstrated in Sect. 6.1.

The use-case presented in Sect. 6.1 demonstrates how changes in the instance level are automatically reflected by the performance indicators in the type model.

This is accomplished by combining the ability to integrate type objects and their instances (e.g., by using the method `getInstances()`—see Sect. 5.2) with the use of change listeners between the levels (as discussed in Sect. 5.3). Thus, requirement *CR5* is satisfied.

In order to support analysis of performance indicators, *CR6* stipulates that it should be possible to drill-down from the BP type model to the source of the problem, i.e., to search for problematic instances. The proposed solution supports this requirement: every OrgML concept inherits from MML:Entity and thus provides its instances (M1 level) with the ability to navigate through their instances (M0 level).

Satisfying requirements *CR3–CR6* guarantees that *GR3* is satisfied, as together they enable monitoring the execution of BPs and analyzing their performance in real-time. In particular, requirements *CR3–CR5* ensure that executed BP instances are immediately reflected both in instance and type models, without requiring any prior preparation, i.e., the transition from the execution phase to the analysis and evaluation phase becomes transparent.

Finally, requirement *CR7* is aimed at using similar BP visualizations throughout the BPM lifecycle by reusing the BP type model visualizations also for capturing instance level information. The BP runtime (type and instance) models presented in Sect. 5 formally reuse the same graphical representations of business process models. This is demonstrated by Figs. 7, 8, 9, 10 in Sect. 6.1.

To summarize, the proposed solution satisfies all general requirements (*GR1–GR3*) with the exception of *GR2*, which is only partially addressed. Table 1 summarizes the evaluation results.

Finally, the multi-level architecture of the proposed solution not only fosters requirements *GR1–GR4*, but also provides an additional advantage over existing BPM suites: it allows for extending the BP modeling language (or any other DSML) beyond a limited set of generic concepts such as activity and event, to include more specific activity and event (meta) types such as “check order” activity or “web-service” activity, which could then be instantiated into more specific types. The “web-service” activity type, for example, could include technical information regarding the service call at the type level (e.g., WSDL data) and at the instance level (e.g., call status and relevant response data).

6.2.2 Expert panel evaluation

The expert panel evaluation had three main objectives: (1) evaluating the relevance of the requirements; (2) evaluating the artifact; and (3) receiving feedback on further development of the artifact.

The panel included seven participants who work as IT consultants for four different organizations and who have at least five years of experience in process design, development of process oriented information systems, or development of BPM tools (on average participants had 13 years of experience).

The evaluation procedure included the following steps. First, the goals of the research as well as the general requirements were presented. Then, the developed prototype was demonstrated to participants. In particular, an exemplary BP was created, and using the agent described in Sect. 6.1, the process model was enacted

Table 1 Requirement evaluation summary

General Require- ments	<i>GR1</i> : directly design BPs into a runtime system	<i>GR2</i> : the BPMS adapts to BP design change	<i>GR3</i> : Direct transition from BP execution to monitoring and analysis the executions throughout the BPM lifecycle	<i>GR4</i> : Similar BP visualizations throughout the BPM lifecycle
BP runtime model Concrete require- ment	<i>CR1</i> : Direct enact- ment	<i>CR2</i> : Synchronized representations of design-time and runtime	<i>CR3</i> : Representa- tion of BP instance state at runtime	<i>CR4</i> : Representa- tion of BP type state (KPIs) at runtime
Satisfied?	✓	✓*	✓	✓
Comments	* Changes to a BP design are immediately available for new executions. However, adapting instances that are being executed at the time of change requires adding a versioning mechanism and is out of the scope of this paper			

several times to create various instances. The automatic visualization of performance metrics based on the BP instances and the ability to drill down to particular instances were demonstrated as well. In addition, we demonstrated how changes to the process can be immediately executed. After the demonstration, the participants were interviewed.

Due to time constraints of participants, we conducted several face-to-face interviews, each interview included one or two participants. While a group interview has some advantages, it may create pressures to arrive at a consensus. Interviewing a single participant at a time circumvents such pressures. Interview durations ranged from 40–90 min. The interviews were guided by a set of fifteen closed questions (see “[Appendix](#)” for the questionnaire), yet, they were carried out as an open discussion and the participant(s) could add comments and ideas beyond the guiding questions. For each of the requirements, participants were asked three types of questions: whether they agree or not that the requirement is important, whether they agree or not that existing BPM suites (they are familiar with) offer support for this requirement, and whether they agree or not that the prototype fulfills the requirement. Responses were indicated on a Likert scale (highly disagree, moderately disagree, neither, moderately agree, and highly agree). The participants’ comments and responses to the questions were compiled and summarized.

With respect to *GR1* (transparent/effortless transition from design to enactment), all participants agreed (either highly or moderately) on its importance, however the responses to whether or not existing BPM suites support *GR1* were varied: some participants moderately agreed that the tools they use offer such support while other participants moderately disagreed. All participants agreed (highly/moderately) that the prototype fulfills the *GR1*. Two participants did not think that the proposed prototype creates added value to users compared to existing tools, while the rest thought that the proposed solution has advantages over the existing tools (e.g., it facilitates a “cleaner” and “more elegant” transition from design to enactment).

With respect to *GR2* (design changes can be directly enacted), the agreement on the importance of *GR2* ranged from highly to moderately agree. One participant however stressed that design changes should not affect process instances that are being executed, which requires tools for conveniently managing such changes (this is out of the scope of our work). Here again, some participants moderately agreed that the tools they use offer support for *GR2* while other moderately/highly disagreed. All participants agreed (moderately/highly) that the proposed prototype supports the enactment of updated process designs.

With respect to *GR3* (direct transition from BP executions to monitoring and analyzing BP performance), all participants agreed (highly/moderately) that it would be valuable for managers that a BPM suite facilitates monitoring the execution of BPs and analyzing their performance metrics. Four participants stated that the BPM suites they are familiar with are capable of providing managers with dashboard capabilities, however two of them said these capabilities were not used in their organizations. All participants highly/moderately agreed that the proposed prototype supports monitoring BP performance, and enables drilling down to particular

instances. It was also mentioned that by using concrete activity types and instances the proposed prototype is more efficient in calculating BP performance metrics compared to existing tools, which use database queries on general activities.

With respect to *GR4* (similar BP visualizations throughout the BPM lifecycle), all participants except one (highly/moderately) agreed that using similar BP visualization for design and evaluation/monitoring would be useful for managers and would facilitate communication with process analysts. One participant though disagreed that requirement *GR4* is important and argued that different role holders are responsible for the activities of design (process analysts) and the evaluation (managers) and thus shared visualization is not necessary. Participants highly/moderately disagreed that the tools they are familiar with visualize performance using BP models.

All participants highly/moderately agreed that the proposed prototype conveniently uses the same BP visualizations for design, monitoring, and performance evaluation. One participant stated that the proposed prototype should also provide additional views of performance indicators using different charts (e.g., bar and pie charts).

The expert panel evaluation is not without limitations. First, participants were presented with only a short demonstration of the prototype capabilities, and did not have a chance to fully experience its advantages and disadvantages in a real-world setting. Second, different participants may have different backgrounds and past experiences, which may perhaps lead to a variety of opinions.

Nevertheless, we were able to identify consensus among the participants. In general, most participants agreed on the importance of the requirements and agreed that the proposed prototype provides a novel and straightforward approach for addressing the requirements. Some participants stated that while the BPM suites they are familiar with address *GR1* and *GR2* to some extent, the proposed prototype provides some advantages with respect to satisfying these requirements. Most participants agreed that the proposed prototype provides better support for activities of monitoring and performance evaluation of BP executions compared to the tools they are familiar with.

7 Conclusion and outlook

We initiated this paper by arguing that BP runtime models present a valuable research objective for BPM. They promise overcoming the conceptual and technical/tool chasm between design and implementation of business processes on the one side, and execution, monitoring, and analysis of running BPM systems on the other. Following up on this reasoning, we developed an environment for designing and managing BPs. It supports the entire BP lifecycle so that all its phases share one single artifact, the BP runtime model, thereby allowing for a transparent transition between lifecycle phases. To support BP runtime models, we designed a suitable multilevel language architecture that implements MEMO DSMLs in XMF.

The presented approach differs from the traditional approach to integrate workflow models and corresponding execution environments in one crucial aspect: it allows for a common representation of models and code. Traditional programming

languages are restricted to using objects on M0 only. This implies that classes/types that are used to represent workflow types within a modelling environment need to be represented on M0, even though they belong conceptually to M1. Therefore, those “types” cannot be instantiated. Instead, the objects representing these classes/types are used for generating the classes that are used in a runtime environment (e.g., the types of a workflow schema). As opposed to that, XMF allows for an arbitrary number of classification levels, where every class is an object at the same time. As a consequence, our tool for modeling workflows allows to represent objects on the level where they conceptually belong. For example, within the tool, an activity type is represented on M1, and hence, can be instantiated and executed in the same namespace.

The common representation of models and runtime representations has various advantages. Since it allows the objects that represent particular instances and the corresponding classes to reside in the same namespace, a user who inspects a workflow instance can directly navigate to the corresponding model. There is also no need to re-generate the run-time representation after the model was changed, which is a substantial advantage in cases where the run-time representation had been modified, because there is no need for synchronization of model and run-time representation. Furthermore, the metamodel that represents the language specification also resides in the same namespace. Therefore, language extensions (e.g., more specific activity and event meta types) can be used immediately after they were applied. In the traditional paradigm that would not only require changing the modeling language in the modeling tool, but also the workflow specification language that is used for run-time purposes.

In addition, the modeling environment is suitable for BP monitoring and evaluation. On the one hand, it is capable of visualizing and navigating to process runtime (instance) models in order to devise a model-based support of process monitoring tasks. On the other hand, the tool enhances BP type models with information that is aggregated from the actual runtime system, thereby turning the BP model into a managerial dashboard to facilitate BP analysis.

Besides integration of concepts at different levels of abstraction, our approach further enables business process instance models to formally reuse graphical representations of business process type models, thereby facilitating BP communication and interpretation by business users. In addition, the use of MEMO OrgML enables enriching BP models with various concepts of other enterprise perspectives that are covered by the MEMO framework, such as business goals, IT resources, risks, and costs.

While our approach focuses mainly on widely automated processes, it also covers user interaction activities (e.g., activities that require users to enter data input) as well as entirely manual activities—for which the user/agent notifies on activity’s start and termination. In addition, BPMS based on the proposed approach could in principle be accessed from other organizations through a shared distributed infrastructure; however, aspects such as access control and multi-user access are out of the scope of our research.

Ultimately, our results may pave the way to a BP modeling environment that supports design and implementation of process models, as well as monitoring, controlling, and analyzing them in an integrated, BP runtime model oriented tool.

However, further research work is required to arrive at a comprehensive BPM environment. For instance, so far, we only used business process instances that were initiated by an agent that imitated the occurrence of external events. While this was adequate for demonstrating the validity of the developed MEMO-XMF environment, in the future we intend to enhance the environment with interfaces for users and external software applications and turn our modeling tool into a full-fledged integrated BPM tool.

Further work is also required for supporting changes to BP designs that occur during the execution of the BP. As discussed in Sect. 6, such situations and, in general, the ability to manage different versions of BP runtime models and their instances require developing a versioning mechanism for BP models.

Based on the expert panel, there is also a need to investigate and test further kinds of visualization. Even though visualization is not at the core of our research, future work should focus on more abstract representations of visualization that allow for individual adaptations.

Finally, runtime models may have to deal with BP instances that unexpectedly deviate from the type model specification. Thus, another prospect is to focus on how to resolve such conflicts. Fortunately, XMF includes several built-in features that provide convenient grounds for supporting instance level deviations from BP type specifications as well as for providing versioning mechanisms.

Appendix: Interview questions

1. It is important for a BPM suite to enable using BP models to directly enact instances and to allow a transparent transition from design to the runtime phase (*GR1*)

Agree						Disagree
	Highly	Moderately	Neither	Moderately	Highly	

2. Currently the BPM suite used in my organization supports a transparent transition from design to runtime

Agree						Disagree	irrelevant ☐
	Highly	Moderately	Neither	Moderately	Highly		

3. OrgML enables to directly design BP models to a runtime system and allows a transparent transition from design and runtime phase

Agree						Disagree
	Highly	Moderately	Neither	Moderately	Highly	

4. It is important that a BPM suite would enable to change BP models at runtime (*GR2*)

Agree						Disagree
	Highly	Moderately	Neither	Moderately	Highly	

5. Currently the BPM suite used in my organization enables changing BP models at runtime (*GR2*)

Agree						Disagree	irrelevant ☐
	Highly	Moderately	Neither	Moderately	Highly		

6. OrgML enables to change the design at runtime

Agree						Disagree
	Highly	Moderately	Neither	Moderately	Highly	

7. It is valuable for me that a BPM suite would enable to monitor the execution of BPs (e.g., current state of execution, completed tasks, involved employees and IT resources) at runtime (*GR3*)

Agree						Disagree
	Highly	Moderately	Neither	Moderately	Highly	

8. Currently the BPM suite used in my organization enables me to easily monitor the performance of BPs (*GR3*)

Agree						Disagree	irrelevant ☐
	Highly	Moderately	Neither	Moderately	Highly		

9. OrgML enables me to easily monitor the performance of BPs (*GR3*)

Agree						Disagree
	Highly	Moderately	Neither	Moderately	Highly	

10. It is valuable for me that a BPM suite would enable to analyze their performance (e.g., durations and costs of different activities and probabilities of different events) at runtime (*GR3*)

Agree						Disagree
	Highly	Moderately	Neither	Moderately	Highly	

11. Currently the BPM suite used in my organization enables me to easily drill down to particular BP instances for further investigation (*GR3*)

Agree						Disagree	irrelevant ☐
	Highly	Moderately	Neither	Moderately	Highly		

12. OrgML enables me to easily drill down to particular instance for further investigation (*GR3*)

Agree						Disagree
	Highly	Moderately	Neither	Moderately	Highly	

13. Using similar process visualizations throughout the BP lifecycle would be valuable for managers (*GR3*)

Agree						Disagree
	Highly	Moderately	Neither	Moderately	Highly	

14. Currently the BPM suite used in my organization uses similar process visualizations throughout the lifecycle (*GR3*)

Agree						Disagree	irrelevant ☐
	Highly	Moderately	Neither	Moderately	Highly		

15. OrgML's uses the same (model-based) visualizations throughout the lifecycle (*GR3*)

Agree						Disagree
	Highly	Moderately	Neither	Moderately	Highly	

16. Can you think of additional requirements for future development that are not supported currently by existing suites / the proposed prototype?

References

- Adams M, Hofstede AHM, Edmond D, Aalst WMP (2006) Worklets: a service-oriented implementation of dynamic flexibility in workflows. In: Meersman R, Tari Z (eds) On the move to meaningful internet systems: OTM confederated international conferences, CoopIS, DOA, GADA, and ODBASE 2006, Montpellier, France. Proceedings, Part I, vol 4275. Springer, Berlin, pp 291–308
- Amoui M, Derakhshanmanesh M, Ebert J, Tahvildari L (2012) Achieving dynamic adaptation via management and interpretation of runtime models. *J Syst Softw* 85(12):2720–2737. <https://doi.org/10.1016/j.jss.2012.05.033>
- Atkinson C, Kühne T (2001) The essence of multilevel metamodeling, vol 2185. Springer, Berlin, pp 19–33
- Atkinson C, Kühne T (2008) Reducing accidental complexity in domain models. *Softw Syst Model* 7(3):345–359
- Bandara W, Indulska M, Chong S, Sadiq S (2007) Major issues in business process management: an expert perspective. In: ECIS
- Basin D, Klaedtke F, Müller S (2010) Policy monitoring in first-order temporal logic. In: Touili T, Cook B, Jackson PB (eds) Computer aided verification: 22nd international conference, CAV 2010, Edinburgh, UK, July 15–19, 2010: proceedings, vol 6174. Springer, Berlin, pp 1–18
- Bencomo N (2009) On the use of software models during software execution. 2009 ICSE Workshop on Modeling in Software Engineering: (MiSE 2009): May 17–18, 2009. IEEE, Piscataway, pp 62–67
- Blair G, Bencomo N, France RB (2009) Models@ runtime. *Computer* 42(10):22–27. <https://doi.org/10.1109/MC.2009.326>
- Briand LC, Williams CE (eds) (2005) Model driven engineering languages and systems: 8th international conference, MoDELS 2005, Montego Bay, Jamaica, October 2–7, 2005: proceedings. Lecture notes in computer science, vol 3713. Springer, Berlin
- Capra L, Cazzola W (2012) Trying out reflective petri nets on a dynamic workflow case. In: Abu-Taieh EMO, El Sheikh AAR (eds) Handbook of research on discrete event simulation environments: Technologies and applications. Information Science Reference, Hershey PA, pp 218–233
- Clark T, Sammut P, Willans J (2008) Applied metamodeling: a foundation for language driven development, 2nd edn. Ceteva

- Der Aalst Van, Wil MP, Hofstede Ter, Arthur HM (2005) YAWL: yet another workflow language. *Inf Syst* 30(4):245–275
- Eclipse (2017) Eclipse modelling project. <http://www.eclipse.org/modeling/>. Accessed 14 June 2013
- Ellis C, Keddara K, Rozenberg G (1995) Dynamic change within workflow systems. In: Conference on organizational computing systems, pp 10–21
- Frank U (2006) Towards a pluralistic conception of research methods in information systems research
- Frank U (2011a) MEMO organization modelling language (2): Focus on Business Processes: ICB-Report 49
- Frank U (2011b) The MEMO meta modelling language (MML) and language architecture: ICB-Report 43
- Frank U (2012) Multi-perspective enterprise modeling: foundational concepts, prospects and future research challenges. *Softw Syst Model*. <https://doi.org/10.1007/s10270-012-0273-9>
- Frank U (2014) Multilevel Modeling. *Bus Inf Syst Eng* 6(6):319–337. <https://doi.org/10.1007/s12599-014-0350-4>
- Frank U, Heise D, Kattenstroth H, Ferguson D, Hadar E, Waschke M (2009) ITML: a domain-specific modeling language for supporting business driven IT management. In: Tolvanen J-P, Rossi M, Gray J, Sprinkle J (eds) Proceedings of the 9th OOPSLA workshop on domain-specific modeling (DSM'09). Helsinki Business School, Helsinki
- Garlan D, Schmerl B (2004) Using architectural models at runtime: research challenges. In: Oquendo F, Warboys BC, Morrison R (eds) Software architecture: first European workshop, EWSA 2004, St. Andrews, UK, May 21–22, 2004: proceedings, vol 3047. Springer, Berlin, pp 200–205
- Georgas JC, van der Hoek A, Taylor RN (2009) Using architectural models to manage and visualize runtime adaptation. *Computer* 42(10):52–60. <https://doi.org/10.1109/MC.2009.335>
- Goldstein A, Frank U (2016) Components of a multi-perspective modeling method for designing and managing IT security systems. *Inf Syst E-Bus Manage* 14(1):101–140. <https://doi.org/10.1007/s10257-015-0276-5>
- Gonzalez-perez C, Henderson-Sellers B (2007) Modelling software development methodologies: a conceptual foundation. *J Syst Softw* 80(11):1778–1796. <https://doi.org/10.1016/j.jss.2007.02.048>
- Habermas J (1984) The theory of communicative action, vol 1. Beacon Press, Boston
- Hilty M, Pretschner A, Basin D, Schaefer C, Walter T (2007) Monitors for Usage Control. In: Etalle S, Marsh S (eds) Trust management, vol 238. Springer, US, Boston, pp 411–414
- Hofstadter DR (1980) Godel, Escher, Bach: an eternal golden braid. Basic Books, New York
- Hull R, Nezhad HRM (2016) Rethinking BPM in a cognitive world: transforming how we learn and perform business processes. In: La Rosa M, Loos P, Pastor O (eds) BPM
- IBM FileNet Case Analyzer (2016) https://www.ibm.com/support/knowledgecenter/en/SSNW2F_5.2.1/com.ibm.p8.ce.admin.tasks.doc/bpfad043.htm
- IBM Business Monitor (2017) <https://www.ibm.com/kn-en/marketplace/business-monitor>. Accessed 08 July 2017
- Ko RKL, Lee SSG, Wah Lee E (2009) Business process management (BPM) standards: a survey. *Bus Process Manag J* 15:744–791. <https://doi.org/10.1108/14637150910987937>
- Kühne T (2006) Matters of (meta-) modeling. *Softw Syst Model* 5(4):369–385. <https://doi.org/10.1007/s10270-006-0017-9>
- Kühne T, Schreiber D (2007) Can programming be liberated from the two-level style. In: Gabriel RP (ed) Proceedings of the 22nd annual ACM SIGPLAN conference on object-oriented programming systems and applications, vol 42. ACM, New York, p 229
- Lehmann G, Blumendorf M, Trollmann F, Albayrak S (2011) Meta-modeling Runtime Models. In: Hutchison D, Kanade T, Kittler J, Kleinberg JM, Mattern F, Mitchell JC, Naor M, Nierstrasz O, Pandu Rangan C, Steffen B, Sudan M, Terzopoulos D, Tygar D, Vardi MY, Weikum G, Dingel J, Solberg A (eds) Models in software engineering, vol 6627. Springer, Berlin, pp 209–223
- Ly LT, Maggi FM, Montali M, Rinderle-Ma S, van der Aalst WM (2015) Compliance monitoring in business processes: functionalities, application, and tool-support. *Inf Syst* 54:209–234. <https://doi.org/10.1016/j.is.2015.02.007>
- Meersman R, Tari Z (eds) (2006) On the move to meaningful internet systems: OTM confederated international conferences, CoopIS, DOA, GADA, and ODBASE 2006, Montpellier, France. Proceedings, Part I. Lecture notes in computer science, vol 4275. Springer, Berlin
- OASIS (2007) Web services business process execution language (WS-BPEL) Version 2.0. <http://docs.oasis-open.org/wsbpel/2.0/OS/wsbpel-v2.0-OS.html>. Accessed 16 June 2017

- Object Management Group (2011) Business process model and notation (BPMN) version 2.0. <http://www.omg.org/spec/BPMN/2.0/>. Accessed 8 July 2017
- Odell JJ (1994) Power types. *Object Orient Program* 7(2):8–12
- Overbeek S, Frank U, Köhling C (2015) A language for multi-perspective goal modelling: challenges, requirements and solutions. *Comput Stand Interf* 38:1–16. <https://doi.org/10.1016/j.csi.2014.08.001>
- Qiu Z, Wong Y (2007) Dynamic workflow change in 5PDM6 systems. *Comput Ind* 58(5):453–463. <https://doi.org/10.1016/j.compind.2006.09.014>
- Rangiha ME, Comuzzi M, Karakostas B (2015) Role and task recommendation and social tagging to enable social business process management. In: Gaaloul K, Schmidt R, Nurcan S, Guerreiro S, Ma Q (eds) *Enterprise, business-process and information systems modeling: 16th International Conference, BPMDS 2015, 20th International Conference, EMMSAD 2015, held at CAiSE 2015, Stockholm, Sweden, June 8–9, 2015, Proceedings*, vol 214. Springer, Cham, pp 68–82
- Reichert Manfred, Dadam Peter (1998) ADEPT flex—supporting dynamic changes of workflows without losing control. *J Intell Inf Syst* 10:93–129
- Rorty R (1999) *Philosophy and social hope*. Penguin Books, New York
- Scanchez M, Barrero I, Villalobos J, Deridder D (2008) An execution platform for extensible runtime models. In: Bencomo N, Blair G, France R, Muñoz F, Jeanneret C (eds) *3rd Workshop on Models@run.time at MODELS: Technical Report COMP COMP-005-2008 Lancaster University*
- Schelp J, Winter R (2006) Method engineering: lessons learned from reference modeling. In: Chatterjee S, Hevner A (eds) *Proceedings of the first international conference on design science research in information systems and technology (DESRIST 2006)*, pp 555–575
- Schonenberg H, Mans R, Russell N, Mulyar N, Der Aalst Van, Wil MP (2008) Towards a taxonomy of process flexibility. *CAiSE Forum* 344:81–84
- Song H, Huang G, Chauvel F, Xiong Y, Hu Z, Sun Y, Mei H (2011) Supporting runtime software architecture: a bidirectional-transformation-based approach. *J Syst Softw* 84(5):711–723. <https://doi.org/10.1016/j.jss.2010.12.009>
- Spinellis D (2008) Rational Metaprogramming. *IEEE Softw* 25(1):78–79
- van der Aalst WMP (ed) (2011) *Process Mining*. Springer, Berlin Heidelberg
- van der Aalst WMP (2013) Business process management: a comprehensive survey. *ISRN Softw Eng* 2013(1):1–37. <https://doi.org/10.1155/2013/507984>
- van der Aalst W, ter Hofstede AHM, Weske M (2003) Business process management: a survey. In: van Aalst Wd, Ter Hofstede A, Weske M (eds) *Business process management: International conference, BPM 2003, Eindhoven, the Netherlands, June 26–27, 2003: proceedings*. Springer, Berlin, New York, pp 1–12
- Weske M (2001) Formal foundation and conceptual design of dynamic adaptations in a workflow management system. In: Sprague RH (ed) *Proceedings of the 34th annual Hawaii international conference on system sciences: Abstracts and CD-ROM of full papers: January 3–6, 2001, Maui, Hawaii*. IEEE Computer Society Press, Los Alamitos, Calif, p 10
- Weske M (2012) *Business process management: concepts, languages, architectures*, 2nd edn. Springer, Berlin
- WfMC (2012) XML process definition language (XPDL). [http://www.xpdl.org/standards/xpdl-2.2/XPDL%202.2%20\(2012-08-30\).pdf](http://www.xpdl.org/standards/xpdl-2.2/XPDL%202.2%20(2012-08-30).pdf). Accessed 08 July 2017